

Fase 4 – Custom Hooks, Deploy y Documentación Final

Descripción General

En esta fase se realizó el cierre del proyecto mediante la creación de custom hooks reutilizables, el despliegue completo de la aplicación en servicios de producción y la elaboración de la documentación final.

El objetivo principal fue transformar la aplicación desarrollada localmente en una aplicación web accesible desde internet, además de mejorar la reutilización de código mediante hooks personalizados y documentar todas las decisiones tomadas durante el desarrollo.

Custom Hooks Implementados

useLocalStorage

Este hook fue creado para encapsular la lógica de lectura y escritura en LocalStorage.

Antes de implementarlo, el acceso a LocalStorage se realizaba directamente desde distintos componentes y contextos. Al extraer esta funcionalidad a un hook reutilizable se logró centralizar la lógica y facilitar su mantenimiento.

Funciones principales:

- Leer valores almacenados.
- Guardar automáticamente cambios.
- Evitar duplicación de código.
- Manejar errores de acceso al almacenamiento.

useFetch

Este hook encapsula la lógica necesaria para realizar peticiones HTTP.

Su implementación incluye:

- Estado de carga.
- Manejo de errores.
- Almacenamiento de datos obtenidos.
- Uso de AbortController para cancelar peticiones cuando un componente se desmonta.

La finalidad de este hook es reutilizar la misma lógica de conexión con APIs sin repetir código en múltiples componentes.

useAtajoTeclado

Este hook permite registrar combinaciones de teclas reutilizables.

Fue utilizado para:

- Alt + N → enfocar el formulario de creación.
- T → alternar entre tema claro y oscuro.

La lógica quedó desacoplada de los componentes y puede reutilizarse fácilmente para nuevos atajos.

useProgresoEntrenamiento

Hook específico del dominio del proyecto.

Su función consiste en calcular estadísticas relacionadas con las sesiones registradas.

Datos calculados:

- Sesiones completadas.
- Minutos entrenados.
- Categorías activas.
- Sesiones pendientes.

Este hook permite reutilizar la misma lógica estadística sin duplicar cálculos en diferentes componentes.

Despliegue del Backend

El backend fue desplegado utilizando Render.

URL pública:

<https://proyecto-final-x9iz.onrender.com>

Características:

- API Express publicada en producción.
- Base de datos SQLite funcional.
- Configuración de variables de entorno.
- Configuración de CORS para aceptar peticiones del frontend desplegado.

Endpoints principales:

- GET /api/items

- POST /api/items
- PUT /api/items/:id
- DELETE /api/items/:id
- POST /api/items/:id/registro
- GET /health

Despliegue del Frontend

El frontend fue desplegado utilizando Vercel.

URL pública:

<https://appm-web.vercel.app>

Configuraciones realizadas:

- Conexión del repositorio GitHub.
- Root Directory configurado para frontend.
- Variable de entorno VITE_API_URL apuntando al backend desplegado en Render.
- Compilación automática con Vite.

Integración Frontend – Backend

Durante el despliegue se realizó la conexión definitiva entre el frontend y el backend.

Para ello se utilizó:

VITE_API_URL

que permite que la aplicación cambie automáticamente entre desarrollo local y producción.

Además se configuró CORS para permitir únicamente peticiones provenientes del dominio desplegado en Vercel.

Mejoras Personales del Proyecto

Además de los requerimientos obligatorios del curso, se inició el desarrollo de una funcionalidad adicional basada en un calendario interactivo.

La idea principal consiste en transformar el calendario en un centro de consulta de información temporal.

Objetivos de esta mejora:

- Seleccionar fechas específicas.
- Consultar actividades realizadas ese día.
- Visualizar estadísticas diarias.
- Mostrar gráficas relacionadas únicamente con la fecha seleccionada.
- Diferenciar entre actividades realizadas y actividades planificadas.

Actualmente se encuentra implementada la estructura base para continuar expandiendo esta funcionalidad.

Mi Gráfica Original

La gráfica original implementada fue:

Minutos entrenados por categoría

Esta gráfica muestra la cantidad total de minutos acumulados en cada categoría de entrenamiento registrada por el usuario.

La razón de elegir esta visualización fue que permite identificar rápidamente en qué áreas se invierte más tiempo de entrenamiento y detectar posibles desequilibrios entre categorías como fuerza, cardio o flexibilidad.

Esta información complementa las demás estadísticas generales y aporta una visión más clara de los hábitos de entrenamiento.

Mis 3 Decisiones Técnicas

1. Uso de un modo híbrido (LocalStorage y API)

Se implementó un StorageProvider capaz de trabajar tanto con LocalStorage como con una API remota.

Esto permitió desarrollar la aplicación incluso cuando el backend aún no estaba disponible y posteriormente conectar ambas partes sin modificar los componentes principales.

2. Centralización del estado mediante useReducer

Se decidió utilizar useReducer para gestionar sesiones, filtros y acciones de la aplicación.

Esto permitió mantener una estructura más organizada y escalable que múltiples useState independientes.

3. Uso de visualización de datos

Se implementaron gráficas dinámicas con Recharts para representar información relevante del entrenamiento.

Esto permitió transformar datos numéricos simples en información visual más fácil de interpretar para el usuario.

Hooks Utilizados

Hook	Archivo	Función
useLocalStorage	src/hooks/useLocalStorage.js	Persistencia de datos en LocalStorage
useFetch	src/hooks/useFetch.js	Peticiones HTTP con manejo de estados
useAtajoTeclado	src/hooks/useAtajoTeclado.js	Gestión de atajos de teclado
useProgresoEntrenamiento	src/hooks/useProgresoEntrenamiento.js	Cálculo de estadísticas del entrenamiento

Stack Tecnológico

Frontend:

- React
- Vite
- Context API
- useReducer
- Recharts

Backend:

- Node.js
- Express
- SQLite

Despliegue:

- Vercel
- Render

Control de versiones:

- Git
- GitHub

Capturas Recomendadas

Para complementar esta documentación se recomienda incluir las siguientes capturas:

1. Vista principal de la aplicación con estadísticas y sesiones.
2. Vista del calendario.
3. Vista con gráficas visibles.
4. Aplicación funcionando en modo API utilizando el backend desplegado.

Sobre Mí

Mi nombre es Abner Gabriel Mejía y actualmente estudio Ingeniería en Ciencias de la Computación en la Universidad del Valle de Guatemala.

Durante el desarrollo de este proyecto pude reforzar conocimientos relacionados con React, manejo de estado global, desarrollo de APIs con Express, persistencia de datos con SQLite y despliegue de aplicaciones web en servicios de producción.

La parte que más me interesó fue la implementación de gráficas y la integración entre frontend y backend mediante una arquitectura híbrida que permite trabajar tanto con LocalStorage como con una API real. Este proyecto me permitió comprender mejor el ciclo completo de desarrollo de una aplicación web moderna.